

User manual - Llama Desktop Launcher

Version 0.1.1 · English

1. What the application does

Llama Desktop Launcher installs or locates the `llama.cpp` engine, finds local GGUF models, and opens the `llama-server` interface in your browser. You can select local-only, LAN, or Tailscale access. Network access requires an API key. Traffic to other computers travels over the selected network.

The app installer **does not include models**. The official engine is downloaded on first use, or you can point the app to an existing installation. The tested engine version is `b11193`.

The interface has three screens: **Llama** for chat and starting the server, **Models** for finding GGUF files and viewing recommendations, and **Settings** for preparing the engine, folders, and profiles. Settings remains available in the navigation on every screen.

2. Requirements

System	Package	GPU backend
Windows x64	Llama Desktop Launcher Setup 0.1.1.exe	Vulkan with a compatible display driver
Linux x64/arm64	ApplImage, DEB, or RPM	Vulkan with compatible drivers and libraries
macOS Intel/Apple Silicon	DMG	Metal

Required disk space depends on the model. Allow room for the GGUF, cache, and engine. Large models can require substantially more RAM or VRAM than the file size alone suggests. Without a compatible GPU, inference may run on the CPU and be slow. The official Linux engine package is built on Ubuntu; other distributions may require compatible system libraries. The GitHub Actions matrix is configured for Windows x64, Linux x64/arm64, and macOS Intel/Apple Silicon, but Linux and macOS artifacts are validated only after their jobs run and the packages are opened on their target systems.

3. Install the application

Windows

1. Open [Llama Desktop Launcher Setup 0.1.1.exe](#).
2. Choose the application installation folder. This is a per-user installer and does not require using [Program Files](#).
3. Select whether to create a desktop shortcut.
4. Finish installation and open the app from the Start menu or desktop shortcut.

Linux

- **DEB**: install with your Debian/Ubuntu compatible package manager.
- **RPM**: install with your Fedora/RHEL compatible package manager.
- **ApplImage**: mark the file executable and run it. ApplImage is portable and remains wherever you place it; a package manager does not install it.

DEB and RPM use the standard locations chosen by the distribution and package manager. Model folders are selected inside the application.

For the current version, use the DEB, RPM, or Appliance package for your system. Experimental portable archives from earlier versions do not replace those packages.

macOS

1. Open the DMG.
2. Drag the app to [Applications](#), or another folder where you want to keep it.
3. Open the app from Finder.

Unsigned development builds may require an explicit exception in macOS security settings. Do not disable system protections globally.

4. First use: set up llama.cpp

On first launch, **Settings** opens automatically if the engine is not ready, there are no models, or no selected model has been saved. After preparing the engine and selecting a model, later launches open **Llama**. You can return to **Settings** from the navigation at any time. The screen explains the remaining setup steps, which you can complete later.

In **Motor llama.cpp**, choose one of these options:

1. **Baixar motor oficial / Download official engine:** the app downloads release [b11193](#) directly from [ggml-org/llama.cpp](#) on GitHub. Windows/Linux use Vulkan; macOS uses Metal. The engine is stored in your user data folder, outside the app installation folder.
2. **Escolher pasta existente / Choose existing folder:** select a folder containing [llama-server](#) (or [llama-server.exe](#)). This lets you reuse an existing installation, including the [b11193-vulkan](#) directory used by the earlier Windows launcher.

Downloading requires internet access. If the release has no package for your CPU architecture, select a compatible engine you built or installed yourself. The app checks SHA-256 when the GitHub release API provides a digest. You can replace a selected engine later by choosing a different existing folder or downloading the official package again.

If you used the previous Windows launcher, first select the folder containing your GGUF files. Under **Configurações > Perfis do launcher anterior / Settings > Previous launcher profiles**, click **Importar perfis TSV / Import TSV profiles** and select [launcher-profiles.tsv](#) from the old launcher folder. Import matches the model's full path; if you moved the GGUF files, select the original folder or adjust profiles manually. Existing profiles in this app are preserved. The result reports imported, missing, already existing, and invalid profiles.

5. Model folders and GGUF files

The **Settings** screen supports **two model locations**. The first defaults to `~/Models/GGUF`; click **Alterar / Change** to select a different folder or drive. The second location is optional and can be added or removed. You can change both later. The **Models** screen scans subfolders for [.gguf](#) files. Choose the model you want to use there; the app saves the selection and settings associated with that file path.

Even before you download a model, **Models** shows estimates based on available RAM, CPU, detectable GPU/VRAM, and free space in the model folders. These ranges are conservative starting points, not guarantees of compatibility or performance. Always check the GGUF file size, license, author requirements, and available disk space. For MoE models, consider the total weight size, not just the active parameters.

To get a model:

1. In **Models**, choose **MoE Q4_K_M** for quantized MoE models, **Denso Q4_K_M** ("Dense Q4_K_M") for quantized dense models, or **Completo F16** ("Full F16") for unquantized files. The filters include GGUF and llama.cpp; the Q4_K_M filters also restrict results to quantized models. You can also open the **MoE Q4_K_M models**, **dense Q4_K_M models**, and **full F16 models** directly.
2. Choose a repository. Review the **model license**, RAM/VRAM requirements, quantization, and the author's instructions.
3. Download a **.gguf** file into one of your configured model folders. This version opens Hugging Face in your browser; your browser handles the file download.
4. Return to the app and refresh the model list. In **Models**, review the hardware recommendation and select the GGUF you want.

For a multimodal model, keep exactly one **mmproj*.gguf** file beside the model; the app will pair it automatically. For split GGUF models, keep all shards in the same folder. The list displays only the first shard.

Note: GGUF is a file format, not a guarantee that every model works with this llama.cpp version. Some repositories may require a newer engine.

6. Llama screen: start a conversation

1. Choose a model detected in your configured folders from **Modelo GGUF** ("GGUF model"). If no selected model has been saved yet, the app opens **Configurações** ("Settings"); first download or locate a model from **Modelos** ("Models").
2. To chat inside the app, click **Conversar aqui** ("Chat here"), enter a message, and send it. Enter sends; Shift+Enter inserts a line break. Chat history is not saved.
3. Click **Abrir no navegador** ("Open in browser") to start **llama-server**, wait for the model to load, and open its web interface. The screen shows the access address and configured sessions.
4. When finished, click **Parar servidor** ("Stop server") in the app.

Use **Abrir CLI no terminal** ("Open CLI in terminal") to launch **llama-cli** (**llama-cli.exe** on Windows) in a separate terminal with the selected model and current settings. Chat and exit from that terminal. Stop the server or integrated chat before starting the external CLI.

Large models may take several minutes to load. Expand the engine log near the bottom of the window to inspect messages or errors. Closing the browser tab does not stop the server. Use **Stop server** or quit the app.

7. Advanced settings

Network links and simultaneous sessions

Under **Acesso e sessões / Access and sessions**, select **Somente este computador / This computer only** or a detected LAN/Tailscale address. Choose **1 to 8 simultaneous sessions**. This sets **--parallel** on **llama-server**; sessions share the available context and memory, so start with 1 or 2 for a large model. This setting controls concurrency, while **Context / Contexto (-c)** controls the token context size.

For LAN/Tailscale, use **Mostrar chave / Show key**, **Copiar chave / Copy key**, or **Nova chave / New key**. Stop the server before rotating the key. The app opens the local browser interface through an authenticated proxy. **Copiar link da API / Copy API link** gives you the **/v1** endpoint. **Copiar link da interface / Copy interface link** gives you the network web address. Remote API clients must send **Authorization: Bearer YOUR_KEY**; the link alone does not grant access. A remote web client must also support this authentication. Never add the key to the URL or publish it in an open message.

The app does not change firewall rules when starting the server. On Windows, while a LAN/Tailscale server is running, enter the **allowed client address or CIDR range** in **Firewall do Windows / Windows Firewall** and click **Criar regra para a porta ativa / Create rule for active port**. Confirm the Windows administrator prompt. The rule is limited to

`llama-server.exe`, the active port, and the entered client range. Check the range before confirming; if the port changes, create another rule. On Linux/macOS, use your distribution's firewall controls. The server selects a free port from 8080 to 8180. Prefer Tailscale or a trusted LAN; direct HTTP on a LAN does not encrypt traffic.

Integrated chat and external CLI

If you prefer the original program, click **Abrir CLI no terminal** ("Open CLI in terminal"). The app opens `llama-cli` (`llama-cli.exe` on Windows) in a separate terminal with the selected model and current settings. Use that terminal to chat and close it; **Encerrar conversa** ("End chat") controls only the integrated chat. Stop the server or integrated chat before opening the external CLI. Linux needs a graphical terminal installed. The external terminal may remain open after the launcher closes.

Advanced settings are in the **Ajustes avançados deste modelo** ("Advanced settings for this model") accordion on the **Llama** screen; it is collapsed initially. Open it to review values, apply an initial suggestion, or compare microbatch. Click **Salvar ajustes deste modelo** ("Save settings for this model") to save the selected GGUF profile. The **Settings** screen also provides the model/profile selector and keeps its settings open for editing. Settings are saved per model path.

Aplicar sugestão inicial ("Apply initial suggestion") fills in fields using RAM, CPU thread count, and GGUF information. The estimate may include GPU/VRAM when the system can detect them. It is a starting point, not a guarantee of speed or exact memory use. Review the values and save the profile when you want to keep them.

Field	llama.cpp flag	Meaning
GPU layers	<code>-ngl</code>	<code>auto</code> lets the engine fit layers; <code>all</code> attempts full GPU offload; <code>0</code> selects CPU.
Device	<code>-dev</code>	Empty means automatic selection. Use an ID such as <code>Vulkan0</code> when several GPUs are present.
Context	<code>-c</code>	Number of tokens retained in a conversation. Higher values use more memory.
MoE layers on CPU	<code>-ncmoe</code>	Keeps expert weights for the first N layers in system RAM. Use only with MoE models; empty disables this setting.
K / V cache	<code>-ctk / -ctv</code>	<code>f16</code> favors fidelity; <code>q8_0</code> uses less memory with a small tradeoff. Test other formats per model.
Flash Attention	<code>-fa</code>	<code>auto</code> , <code>on</code> , or <code>off</code> ; availability depends on backend and model.
CPU threads	<code>-t</code>	Threads used by CPU portions. Empty uses the engine default.
Batch	<code>-b</code>	Logical prompt processing batch size.
Microbatch	<code>-ub</code>	Physical chunk size. It cannot be larger than batch.

The app also passes `-fit on` so the engine can adapt to available memory. Leave `-ncmoe` empty for dense models. For an MoE too large for VRAM, adjust CPU MoE layers while watching RAM use and throughput.

8. Review recommendations and benchmark performance

On the **Models** screen, the app prepares a prompt with available hardware information, the selected model, and its settings. Click **Copiar prompt de análise** (“Copy analysis prompt”) to copy it for use elsewhere, or **Analisar com modelo selecionado** (“Analyze with selected model”) to send it through the integrated chat to the current model. If you do not have a model yet, you can copy the prompt and use the local recommendations; AI analysis becomes available after selecting a GGUF.

The model's response is a recommendation for you to review. It does not automatically change or save settings. Check compatibility, memory, license, and actual measurements before applying changes. Local suggestions are also estimates; the benchmark below measures performance on your computer.

To measure microbatch after selecting and loading a GGUF:

1. On the **Llama** screen, open **Ajustes avançados deste modelo** (“Advanced settings for this model”). To get suggestions for several fields, click **Aplicar sugestão inicial** (“Apply initial suggestion”); this does not run the model.
2. Stop the server or active chat and click **Comparar microbatch** (“Compare microbatch”). [llama-bench](#) tests the current value and another value of up to 128, capped by batch. If they match, it runs only one test.
3. Wait for prompt processing and generation results in tokens per second. The app fills in **Microbatch** with the better value and leaves the other fields unchanged.
4. Review the fields and click **Salvar ajustes deste modelo** (“Save settings for this model”) to keep the result without starting the model.

The benchmark may load the model more than once and can take several minutes. It does not save the profile by itself or optimize context, cache, GPU layers, or MoE layers. You can adjust those fields manually.

9. Troubleshooting

Problem	What to check
No models listed	Confirm the model folders and .gguf extension; click Refresh list .
Engine download fails	Check internet access, free space, and GitHub access; retry or select an existing engine folder.
“Package without llama-server”	The release layout may have changed; select a compatible runtime manually.
Server exits during startup	Read the engine log; lower context/batch, try <code>-ngl auto</code> , or reduce GPU layers.
GPU not used	Update Vulkan drivers on Windows/Linux; on macOS check that you installed a Metal build. Read the engine log.
Benchmark fails	Ensure the runtime folder contains llama-bench ; try more conservative settings.
Model fails to load	Check all shards, b11193 compatibility, model repository instructions, and available memory.

10. Data, updates, and safe uninstalling

Preferences and profiles are stored as `settings.json` in the app's user data folder. The downloaded engine is stored under `engines/` in the same area. Model files stay in the folders you chose. The app does not allow a model folder inside the application installation or its internal user data directory.

Before uninstalling, click **Parar servidor / Stop server** if needed, then close the app normally. On Windows, the uninstaller checks whether the app is still open and asks you to close it; click **Retry** after closing. It does not force the app to quit. Use **Installed apps** in Windows Settings. By default, it removes the app and shortcuts while **preserving models, profiles, and the downloaded engine**. Do not use advanced app-data removal options without inspecting the data folder first.

On Linux, remove DEB/RPM with your package manager, or delete the AppImage file. On macOS, move the `.app` to Trash. User data and GGUF files remain for manual review. If you want to remove them, do so separately after checking the path and backing up `settings.json` if you want to keep profiles. Never delete a model folder simply because it was selected in the app.

11. Credits

`llama.cpp` was started by Georgi Gerganov and is maintained by the [ggml-org contributors](#) under the [MIT license](#). This independent launcher was developed for Célio with Codex, based on the earlier WinForms launcher, and is released under the **MIT license**, with copyright held by Célio. The repository and npm package may remain private for now. The `llama.cpp` authors do not participate in or endorse this app. Each model has its own license.